

Mikrocomputertechnik 1

Einheit 8: I/O & Systemarchitektur

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 84

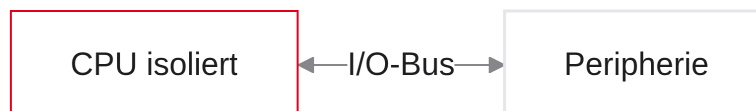
I/O & Systemarchitektur

Folie

2 / 84

Die Brücke zur Außenwelt

- Herzlich willkommen zur achten Vorlesung in Mikrocomputertechnik 1
- Letzte Einheiten: Datenpfad, Steuerwerk, Pipelining
- Unser 5-stufiger RISC-V-Prozessor ist schnell und logisch korrekt
- Letztes Problem: der Chip ist taub und stumm
- Heute: die Brücke zur Außenwelt (Input/Output)

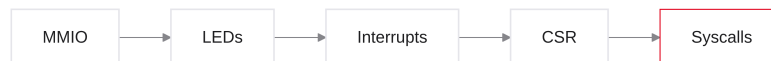


Notizen

Willkommen zu Einheit 8. Datenpfad, Control Unit und Pipelining stehen. Der Chip rechnet isoliert: Ergebnisse bleiben im Silizium. Heute: Kommunikation mit Sensoren, Aktoren und dem Betriebssystem.

Agenda für den heutigen Tag

1. Memory-Mapped I/O — wie spricht die CPU mit Peripherie?
2. Schalter & LEDs — Ansteuerung und Entprellung
3. Polling vs. Interrupts — nicht sinnlos warten
4. Das CSR-System — versteckte Register für Systemzustände
5. Syscalls & OS — Hilfe vom Betriebssystem (`ecall`)
6. Labor (Ripes/Venus) — virtuelle LEDs und Konsolenausgaben



Notizen

Von der untersten Hardware bis zur OS-Abstraktion: LEDs, Polling vs. Interrupt, CSRs und `ecall`.

Rückblick: Der isolierte High-Speed-Chip

- Status aus Einheit 7: Pipeline mit hohem Durchsatz
- ALU, PC und RAM arbeiten fehlerfrei
- Alle Daten bleiben auf dem Die gefangen
- Kein Knopfdruck kann das laufende Programm beeinflussen

Ein Prozessor ohne I/O ist wie ein Taschenrechner ohne Display und ohne Tasten: physikalisch nutzlos.

Notizen

Die schnellste Pi-Berechnung hilft nicht, wenn das Ergebnis für immer in `s0` liegt.

Das Brain-in-a-Jar-Problem

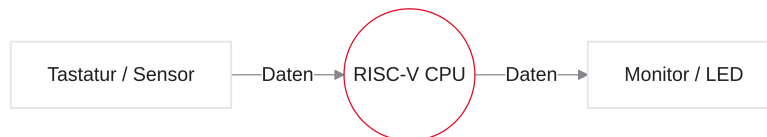
- Gedankenexperiment: Gehirn im Einmachglas
- Rechnet und denkt — ohne Sinnesorgane
- Unsere CPU ist genau dieses Gehirn
- I/O ist das Nervensystem, das wir anschließen
- Reize empfangen, Aktionen in der Welt auslösen

Notizen

I/O = Kamera, Mikrofon und Greifarme für das isolierte Gehirn.

Peripherie (Die Außenwelt)

- Alles außer der reinen Recheneinheit: Peripherie
- Input (Sensoren): Tastatur, Maus, Temperatur, Schalter, Netz (Empfang)
- Output (Aktoren): Monitor, LED, Lautsprecher, Motor, Netz (Senden)
- Frage: Wie anbinden, ohne für jedes Gerät ein neues Steuerwerk?



Notizen

Peripherie verbindet Bits mit Physik. Ziel: ein einheitliches Ansteuerungsmodell für RISC-V.

Memory-Mapped I/O

Wie spricht die CPU mit Peripherie?

- Zwei historische Philosophien:
- Port-Mapped I/O (Komplexität)
- Memory-Mapped I/O (Einfachheit)
- Intel x86: historisch oft Port-Mapped
- ARM und RISC-V: konsequent Memory-Mapped I/O

Notizen

Spezielle I/O-Befehle vs. Illusion „Hardware ist Speicher“ — diese Wege spalten die Architekturen bis heute.

Port-Mapped I/O (x86)

- Separater I/O-Bus und eigene ISA-Befehle: IN, OUT
- Beispiel: IN AL, 0x60 (klassischer Tastatur-Port)
- Nachteile: komplexeres Steuerwerk, begrenzter Port-Raum, mehr Silizium

```
# Alter x86-Stil (Port-Mapped I/O):  
MOV DX, 0x03F8    # COM1  
OUT DX, AL        # Byte aus AL an den Port
```

Notizen

Extra-Leitungen und Sonderbefehle: normale ALU-Befehle greifen auf Ports nicht zu.

Memory-Mapped I/O (MMIO)

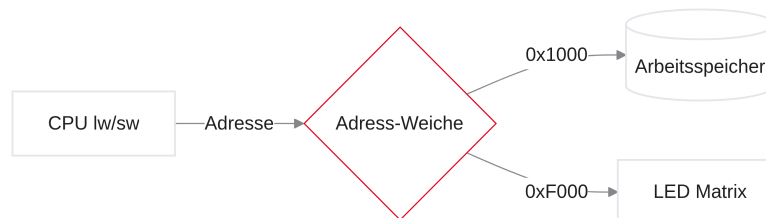
- RISC-V-Weg: kein neuer I/O-Bus, keine neuen Befehle
- Philosophie: Hardware tarnen als Arbeitsspeicher
- Kommunikation wie mit einer Array-Zelle im RAM
- Bekannte Befehle: lw und sw

Notizen

Load/Store reichen: Thermometer und LED erscheinen als 32-Bit-Zellen im Adressraum.

Die Illusion des Speichers

- CPU legt 32-Bit-Adresse auf den Bus — glaubt, sie spricht mit RAM
- Adress-Decoder als Weiche
- Spezielle Adressen (z. B. `0xF0000000`) schalten RAM ab und Peripherie an
- Gerät legt Daten auf den Bus; CPU lädt ahnungslos nach t_0



Notizen

Betrug auf dem Mainboard: gleiche Pins, andere Chip-Select-Ziele.

Der Adressraum (Address Map)

- 32-Bit-Adressraum: $2^{32} = 4\,294\,967\,296$ Bytes (4 GB)
- Feste Address Map im Chip-Handbuch
- Beispielhafte Aufteilung:
 - `0x00000000–0x0FFFFFFF`: Boot-ROM / Code
 - `0x10000000–0x7FFFFFFF`: RAM (Heap, Stack, `.data`)
 - `0xF0000000–0xFFFFFFFF`: MMIO (Sensoren, LEDs)

```
lw aus 0x10000000 -> Array-Element im RAM
lw aus 0xF0000000 -> z.B. Pixel/Sensor-Wert
```

Notizen

Reviere abstecken: niedrige Adressen RAM, hoher Bereich Peripherie.

Der Address-Decoder (Hardware)

- Oft Teil des SoC / der Bridge-Logik
- Jedes Bauteil: Chip-Select (CS)
- CS = 0: Bauteil schläft; CS = 1: aktiv
- Decoder: AND-Netzwerk auf oberen Adressbits (z. B. 0xF . . .)

Notizen

Nur das adressierte Gerät wacht auf und bedient den Datenbus.

Load/Store für I/O

- MMIO macht Bare-Metal-Assembler einfach
- Schalter lesen = lw; Motor starten = sw mit 1

```
li s1, 0xF0000000    # Thermometer-Basis
li s2, 0xF0000010    # Warn-LED

lw t0, 0(s1)         # MMIO-Read
li t1, 100
blt t0, t1, Safe

li t2, 1
sw t2, 0(s2)         # MMIO-Write: Alarm-LED
Safe:
```

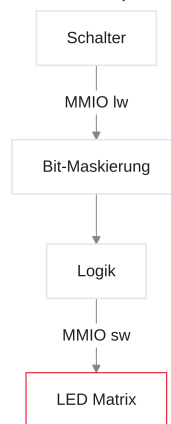
Notizen

Keine neuen Opcodes: Temperatur schwellenbasiert auf LED abbilden.

Praxis der MMIO (Schalter und LEDs)

Von der Theorie zur Laborpraxis

- Input: D-Pad / Schalter
- Einzelne Bits maskieren
- Output: LED-Matrix
- Gefahr: volatile Speicher (Compiler-Caches)



Notizen

Bit-Maskierung aus früheren Einheiten wird wieder gebraucht.

Input: Switches (Bitmaske)

- Beispiel: Switches an `0xF0000000` (Ripes)
- `lw` liefert ein 32-Bit-Wort als **Bit-Map**
- Jedes Bit = ein physikalischer Schalter (1 = an)

Bits 31:4	Bit 3	Bit 2	Bit 1	Bit 0
0...	SW3	SW2	SW1	SW0

Notizen

Nur die Switches-Komponente ist eine Bitmaske. Das D-Pad ist anders aufgebaut (nächste Folie).

Folie

18/84

Input: D-Pad (eigene Register)

- In Ripes: **vier separate 32-Bit-Register**, je Richtung eines
- Offsets vom D-Pad-Basis: UP +0, DOWN +4, LEFT +8, RIGHT +12
- LSB = Tastenzustand (1 = gedrückt); restliche Bits typisch 0
- Ripes-Symbole: D_PAD_0_UP, D_PAD_0_DOWN, ...

Notizen

Nicht mit Switches verwechseln: Beim D-Pad gibt es keine Bitmaske an einer Adresse. Jede Richtung wird separat per `lw` gelesen.

Folie

19/84

Switches auslesen und maskieren

- Problem: mehrere Schalter gleichzeitig (z. B. `0b0101`)
- `beq` auf den Rohwert scheitert an Stör-Bits
- Lösung: Bit isolieren mit `andi` und Maske

```
li s1, 0xF0000000      # Switches-Basis
lw t0, 0(s1)

# Bit 2 (SW2): Maske 4 = 0b0100
andi t1, t0, 4
bne t1, zero, Jump
```

Notizen

Wert 5 bedeutet SW0 und SW2 an. AND filtert auf eine saubere 4 oder 0.

D-Pad auslesen (eine Richtung)

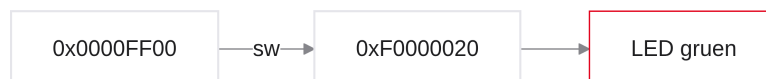
```
li s1, 0xF0000000    # D-Pad-Basis
lw t0, 0(s1)          # UP    (+0)
lw t1, 4(s1)          # DOWN  (+4)
lw t2, 8(s1)          # LEFT  (+8)
lw t3, 12(s1)         # RIGHT (+12)
bne t0, zero, UpPressed
```

Notizen

Prüfen Sie den LSB bzw. den ganzen Wortwert auf $\neq 0$. Nach dem Erkennen: warten, bis die Taste wieder losgelassen ist (Release), sonst rauscht ein Klick durch mehrere Zustände.

Output: Die LED-Matrix

- Beispiel-Adresse 0xF0000020
- sw brennt Bit-Muster auf die Dioden
- RGB oft als 0x00RRGGBB
- 0x00FF0000 → knallrot



Notizen

Hex-Farbe per Store Word: Computergrafik auf unterster Ebene.

Flüchtigkeit von I/O-Registern (Volatile)

- RAM: geschrieben = gelesen (passiv)
- MMIO: lebendig und flüchtig (volatile)
- Schalter-Register ändert sich durch externe Physik — ohne sw der CPU

```
RAM: Tresor - ändert sich nur, wenn ICH schreibe.
I/O: Fenster - die Welt draußen ändert sich ständig.
```

Notizen

Zwei aufeinanderfolgende `lw` können verschiedene Werte liefern.

Folie

23 / 84

Caching-Gefahren bei I/O

- Zwei getrennte Gefahren:
 - Compiler: ohne `volatile` eliminiert → 03 wiederholte Loads
 - Hardware-Cache: MMIO muss in der Memory Map non-cacheable sein
- Bug: Knopfdruck wird nie bemerkt
- In Assembler: jedes `lw` in der Schleife ist ein Bus-Zugriff

```
/* gefährlich ohne volatile: */  
int *button = (int *)0xF0000000;  
while (*button == 0) { }  
  
/* Rettung (Compiler): */  
volatile int *button = (volatile int *)0xF0000000;
```

Notizen

`volatile` zwingt den Compiler zu echten Loads. Separat: SoC/Page-Attribute müssen MMIO als non-cacheable markieren, sonst liefert der Cache veraltete Werte.

Folie

24 / 84

I/O-Module in Ripes (Vorschau)

- Eigener I/O-Tab: virtuelle Boards einstecken
- D-Pad, Switches, LED-Matrizen
- Ripes emuliert den MMIO-Decoder
- Mausklick auf Schalter → Bit im gespiegelten Adressraum

Notizen

Graue Registerzahlen verlassen: Hardware-Komponenten visuell verdrahten.

Das Memory-Map-Setup

- Vor dem Code: Base Address der Peripherie ablesen (z. B. 0xF0000000)
- Falsche Adresse: Schreiben landet im normalen RAM
- Profi-Praxis: Konstanten / BSP-Header

```
#define LED_BASE    0xF0000020  
#define SWITCH_BASE 0xF0000000
```

Notizen

In Ripes die Basisadresse der Komponente prüfen — sonst Heap statt LED.

Feedback-Loop (Das Echo)

- Hello World der Hardware: Schalter spiegelt LED
- Endlosschleife: lw Schalter, sw LED, Jump

```
li s1, 0xF0000000    # Schalter  
li s2, 0xF0000020    # LED  
  
Loop:  
    lw t0, 0(s1)  
    sw t0, 0(s2)  
    j Loop
```

Notizen

CPU als intelligentes Übertragungskabel: Load, Store, Jump.

Entprellung (Debouncing)

- Mechanische Taster prellen (Bouncing)
- Millisekunden Chaos 1/0, bevor stabil 1
- Pipeline fragt milliardenfach ab
- Ein menschlicher Klick wird als Dutzende Klicks gezählt

Notizen

Counter-Variablen explodieren ohne Entprellung.

Software-Debouncing

- Erste 1 erkannt: Druck erkannt
- ~20 ms Signal ignorieren (Kontakte beruhigen)
- Delay-Loop: Takte absichtlich verbrennen

```
li t1, 10000
Delay:
    addi t1, t1, -1
    bne t1, zero, Delay
```

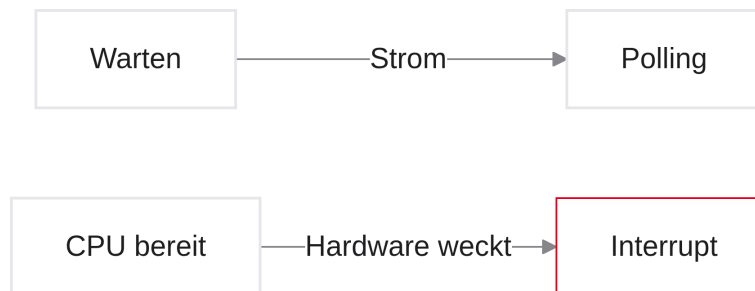
Notizen

Software-Debouncing: nach der ersten Flanke warten, dann sauber weiterlesen.

Polling vs. Interrupts

Aktives Warten vs. Aufwecken

- Ineffizienz im Echo-Design: Busy Waiting
- Polling: Endlosschleife fragt ab
- Interrupt: Hardware weckt die CPU (Türklingel)
- Ausnahmen (Exceptions) vs. Interrupts



Notizen

Phase 3 beginnt: Polling verschwendet Energie. Als Nächstes das Interrupt-System moderner Computer.

Das Problem des Wartens

- Pipeline-CPU z. B. bei 4 GHz: 4 Milliarden Takte pro Sekunde
- Mensch: vielleicht alle 2 Sekunden ein Tastendruck
- Zwischen zwei Klicks: etwa 8 Milliarden Taktzyklen
- Frage: Was tut das Programm in der Zwischenzeit?

Ein Mensch ist für einen Prozessor unvorstellbar langsam.
Wartet die CPU auf uns, vergehen für sie "Jahrzehnte".

Notizen

Hardwarewelt blitzschnell, physische Welt im Vergleich im Koma. Ein Tastendruck ist für die CPU ein Ereignis alle paar "Jahrhunderte". Was macht der Prozessor in den Milliarden Zyklen dazwischen?

Polling (Busy Waiting)

- Naive Lösung: Polling (Abfragen) / Busy Waiting
- Metapher: Kind auf dem Rücksitz — “Sind wir schon da?”
- “Sind wir schon da?” → Nein ... Nein ... Ja!
- CPU fragt in einer Endlosschleife das MMIO-Register ab, bis Bit = 1

Notizen

Geschäftiges Warten: ununterbrochen Load, Prüfen, Branch. Das klassische “Sind wir schon da?”-Syndrom.

Polling in Assembler

- Wie das Echo-Programm: Wait-Schleife, solange Register = 0
- Erst wenn die Hardware Bit 1 setzt, läuft nützlicher Code weiter

```
li s1, 0xF0000000      # Schalter-Adresse

Wait:
    lw t0, 0(s1)         # MMIO-Read
    beq t0, zero, Wait   # noch 0? sofort wieder

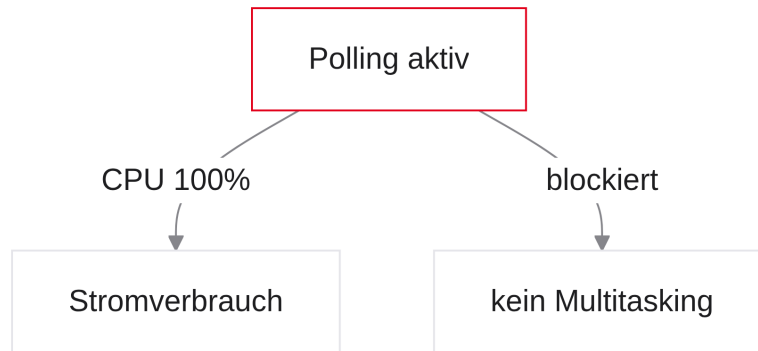
    # Knopf wurde gedrückt - weiter mit Nutzcode
```

Notizen

Load und Branch milliardenfach: sinnlose Abfragen, während der Mensch überlegt.

Die Nachteile von Polling

- Energieverschwendung: CPU unter Volllast, Hitze, leerer Akku
- Keine Nebenläufigkeit: Browser, Audio, UI blockiert
- In der Praxis nur noch Nischen (z. B. Ultra-Low-Latency)



Notizen

Polling saugt den Akku leer und tötet Multitasking. Für moderne Systeme architektonisch tödlich.

Folie

35 / 84

Die Hardware-Lösung: Die Türklingel

- Paket erwarten: nicht stündlich an der Klinke rütteln (Polling)
- Stattdessen: Türklingel einbauen
- Normales Leben weiterführen; erst beim Klingeln reagieren
- Rolle umkehren: Peripherie meldet sich, wenn Daten da sind

Notizen

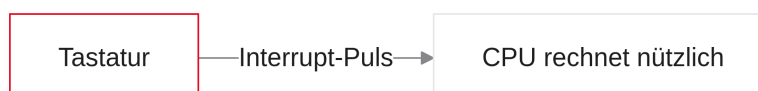
Aktiv warten vs. Ereignis-getrieben: CPU widmet sich nützlicher Arbeit, bis das Gerät signalisiert.

Folie

36 / 84

Einführung von Interrupts

- Türklingel-Prinzip = Interrupt (Unterbrechung)
- Spezielle Pins verbinden Peripherie mit der CPU
- Tastendruck: Stromimpuls auf den Interrupt-Pin
- CPU speichert Kontext, springt zur ISR, kehrt danach zurück



Notizen

Rückgrat moderner Betriebssysteme: asynchrones Aufwecken statt Busy Waiting.

Der Ablauf eines Interrupts

- Main-Programm läuft (z. B. Berechnung / Player)
- Interrupt-Signal trifft den CPU-Pin
- Hardware unterbricht; PC springt zur ISR
- ISR verarbeitet die Eingabe
- `mret`: Rückkehr ins Main-Programm



Notizen

Zeitstrahl: Spiel läuft, Leertaste feuert Interrupt, ISR puffert den Event, `mret` setzt unsichtbar fort.

Exceptions vs. Interrupts

	Exceptions	Interrupts
Quelle	CPU-intern	Hardware-extern
Auslöser	aktueller Befehl	unabhängig vom Code
Timing	synchron	asynchron
Beispiel	Illegal Instr., Access Fault	Timer, Tastendruck

Notizen

Interrupts von außen und asynchron; Exceptions von innen und synchron am fehlerhaften Befehl. Beide unterbrechen — Ursachen verschieden.

Exceptions im Detail

- Illegal Instruction: Decoder kennt den Opcode nicht
- Access Fault: `lw/sw` auf gesperrte/ungültige Adresse (ohne MMU)
- Page Fault: nur mit virtuellem Speicher (Paging)
- Hinweis: Integer-Div/0 trappt in RISC-V nicht (anders als x86)
- OS beendet oft mit Segmentation Fault

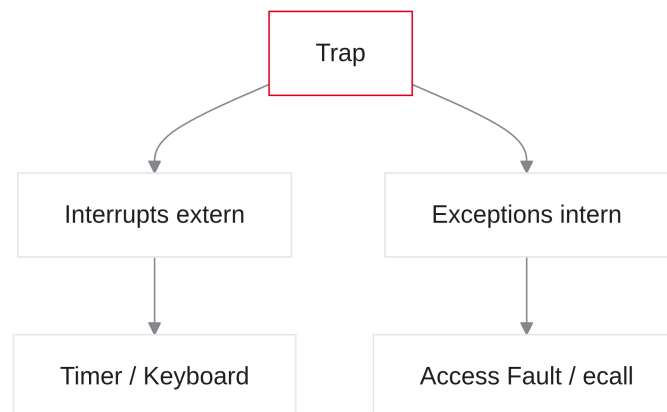
```
int *ptr = NULL;  
int crash = *ptr;    /* typisch: Access Fault */
```

Notizen

Nullpointer: Memory-Unit alarmiert. Bei RISC-V Integer-Division durch 0 entsteht kein Trap (Quotient = all-1s).

Der Regenschirm-Begriff: Traps

- Exceptions und Interrupts teilen denselben Rettungsablauf
- RISC-V-Dachbegriff: Trap (Falle)
- Trap = Unterbrechung des Kontrollflusses ohne Branch/Jump
- "Trap ausgelöst" = Notfall-Ablauf gestartet



Notizen

In der RISC-V-Spezifikation: Trap = jedes erzwungene Verlassen des normalen Programmflusses Richtung OS/Handler.

Was passiert bei einem Trap?

1. Rettungsanker: aktuellen PC merken (Rückkehr)
2. Pipeline flushen: IF/ID hinter dem Ereignis löschen
3. Sprung: PC auf feste Handler-Adresse setzen (ISR)

Ein Trap ist ein erzwungener Hardware-Sprung an eine vom Betriebssystem vorbereitete Adresse.

Notizen

Drei Hardware-Schritte: PC sichern, Pipeline säubern, PC auf Rettungscode setzen.

Die Gefahr des Clobberings

- ISR ist normaler Assembler und nutzt $t0$, $s1$, $a0$, ...
- Trap kommt ohne Vorwarnung mittendrin
- Die ABI gilt weiter — aber es gibt keinen Caller, der Temporaries rettet
- Deshalb: Handler muss jedes genutzte Register selbst sichern

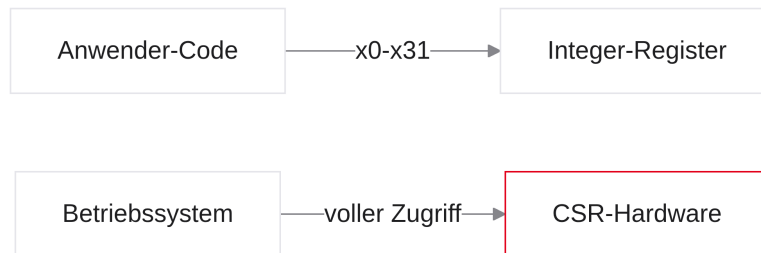
Notizen

Kein Caller bei asynchronen Interrupts: ungesicherte Temporaries zerstören den Kontext. Lösung: Kontextsicherung und CSRs.

Das CSR-System

Schattenregister für die Notaufnahme

- Zustand retten, ohne User-Register zu zerstören
- `x0–x31` sind durch Anwendercode belegt
- Privilegienstufen (Ring-Sicherheit)
- CSRs: unsichtbarer Speicherplatz im Chip
- Befehle `csrr`, `csrw` und der ISR-Ablauf



Notizen

Phase 4: Geheim-Tresor in der CPU, den der User nicht sieht — Fundament sicherer OS-Handler.

Die Beschränkung von RV32I

- RV32I: genau 32 Integer-Register — vom Compiler ausgereizt
- OS braucht Platz für Timer, Trap-Ursache, Puffer — oft vor sicherem Stack-Zugriff
- Teufelskreis: Stack-Sicherung braucht `sp` — dem User-`sp` darf man nicht trauen

Schatten-Register: vertrauenswürdig und nicht manipulierbar.

Notizen

Handler darf User-Registern nicht vertrauen. Deshalb separate, sichere Hardware-Register.

Die CSRs (Control & Status Registers)

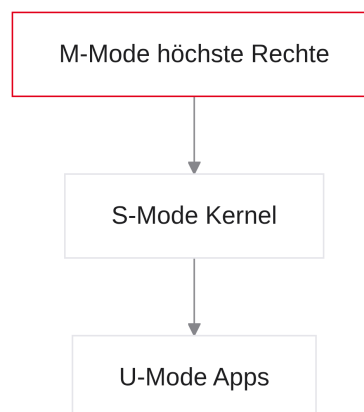
- Extension Zicsr: separater Adressraum, bis zu 4096 CSRs
- Kein RAM/MMIO — Flip-Flop-Bänke im Kern
- Namen z. B. `mstatus`, `mtvec`, `mcause`
- Normale Befehle (`add`, `lw`) greifen nicht zu

Notizen

CSRs sind für User-Code unsichtbar und abgekapselt.

Privilegienstufen (Privilege Levels)

- Machine Mode (M): höchste Rechte, Boot; voller CSR-Zugriff (Firmware)
- Supervisor Mode (S): großes OS (Linux); viele CSRs (z. B. Paging)
- User Mode (U): Apps im Käfig; CSR-Zugriff → Trap



Notizen

Boot mit höchsten Privilegien, dann Herabstufung. Hardware verriegelt CSR-Türen im User Mode.

CSR-Befehle (Zicsr)

- `csrr rd, csr`: CSR → normales x-Register
- `csrw csr, rs1`: x-Register → CSR

```
csrr t0, mstatus      # Status nach t0
li t1, 8              # Maske: Bit MIE
csrrs x0, mstatus, t1 # nur Bits setzen (OR)
```

Notizen

Brücke zwischen Schattenwelt und Integer-Registerbank. `csrw` überschreibt das ganze Register — Bits setzen mit `csrrs`.

Atomare CSR-Befehle

- Gefahr: Interrupt zwischen `csrr` und `csrw` (Race Condition)
- Lösung: `csrrw` — Read und Write in einer Instruktion (atomar)
- Software-seitig ununterbrechbar (eine Instruktion)

```
li t1, 0x1
csrrw t0, mstatus, t1 # t0 = alt; mstatus = t1
```

Notizen

Atomarer Austausch verhindert verlorene Bits zwischen Read und Write.

CSR: mtvec (Trap-Vector)

- Vier Kern-CSRs für Trap-Handling
- `mtvec`: Basisadresse der ISR
- Bei Trap: Hardware kopiert `mtvec` in den PC

```
Boot des OS:
la t0, MeineRettungsRoutine
csrw mtvec, t0
```

Notizen

Zentrale Notaufnahme-Adresse: Hardware springt blind dorthin.

Folie

51 / 84

CSR: mepc (Exception PC)

- Wo wurde der User-Code unterbrochen?
- Hardware kopiert den unterbrochenen PC automatisch nach `mepc`
- Bei synchronen Exceptions/`ecall`: `mepc` zeigt auf den auslösenden Befehl
- Handler muss typischerweise `mepc = mepc + 4` setzen, sonst Endlosschleife

Notizen

`mepc` = Rückflugticket. Ohne +4 kehrt `mret` zum selben `ecall` zurück. Simulatoren wie Venus erledigen das intern.

Folie

52 / 84

CSR: mcause (Cause)

- Warum aufgeweckt? Tastendruck, Access Fault, Syscall?
- Hardware schreibt Fehler-/Ursachencode nach `mcause`
- MSB = 1: Interrupt; MSB = 0: Exception (zuerst prüfen)
- Beispiele: 2 = Illegal Instruction; 8 = Environment Call (U-Mode)

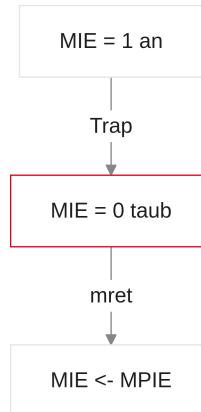
```
csrr t0, mcause
li t1, 8
beq t0, t1, Syscall    # nach Interrupt-Bit-Check
```

Notizen

Erste Handlung der ISR: Interrupt-Bit und Exception-Code lesen, dann abzweigen.

CSR: mstatus (Status)

- Herz der Kontrolle; u. a. Bit MIE (Machine Interrupt Enable)
- MIE = 1: Interrupts wahrnehmen; MIE = 0: taub
- Beim Trap: Hardware setzt MIE automatisch auf 0 (kein Nesting)



Notizen

Hauptschalter der Ohren: Notaufnahme darf nicht durch einen zweiten Interrupt zerrissen werden.

Der Trap-Handler (ISR)

- Schritt 1: Kontextsicherung aller genutzten x-Register
- Echter Kernel: zuerst `sp` mit `mscratch` tauschen (User-`sp` ist unsicher)
- Unten: vereinfachte Lehr-Sequenz (Stack bereits gültig angenommen)

```

Trap_Handler:
    # real: csrrw sp, mscratch, sp
    addi sp, sp, -16
    sw t0, 0(sp)
    sw t1, 4(sp)
    sw a0, 8(sp)
  
```

Notizen

Produktionssysteme tauschen auf den Kernel-Stack. Die Kurzform dient nur der Lesbarkeit.

Kontext-Sicherung und Arbeit

- Schritt 2: `mcause` lesen, Treiber, eigentliche Aufgabe
- Schritt 3: Register spiegelverkehrt wiederherstellen

```
csrr a0, mcause
call Lese_Tastatur

lw t0, 0(sp)
lw t1, 4(sp)
lw a0, 8(sp)
addi sp, sp, 16
```

Notizen

Nach Sicherung freie Bahn; danach Zustand exakt wie vor dem Interrupt.

Die Rückkehr (`mret`)

- Nicht `ret` — in `ra` steht nichts Brauchbares
- Zieladresse liegt in `mepc`
- `mret` (Machine Return) atomar:
 - `mepc` → PC
 - MIE wird aus MPIE wiederhergestellt (oft wieder 1)
 - Privilegien zurück (oft U-Mode)

```
mret    # zurück ins User-Programm
```

Notizen

Bei Trap: `MPIE <- MIE`, `MIE <- 0`. Bei `mret`: `MIE <- MPIE`, plus Rücksprung und Ring-Herabstufung.

Zusammenfassung Trap-Ablauf

- Lebenszyklus jedes Multitasking-OS
- Ein Kern: scheinbar viele Programme + I/O ohne Absturz



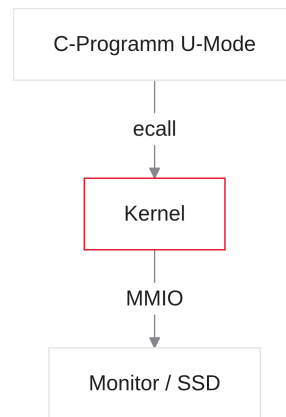
Notizen

Hardware fängt Signale und steuert CSRs; Kernel sichert Kontext und arbeitet. Timer-Interrupts verteilen Rechenzeit.

Syscalls & Das Betriebssystem

Absichtliche Traps und der Pförtner

- Software kann absichtlich einen Trap auslösen
- Mauer zwischen User und Hardware
- OS als Pförtner
- Befehl `ecall` und ABI-Verträge
- Labor: Text auf die Venus-Konsole



Notizen

Phase 5 beginnt: System Calls als Brücke vom User Mode in den privilegierten Kernel.

Folie

60 / 84

Software-Traps

- Hardware kann die CPU per Interrupt unterbrechen
- Kann ein User-Programm absichtlich einen Trap auslösen?
- Ja: wenn C-/Assembler-Code die Hilfe des OS braucht
- Absichtliche Software-Traps = System Calls (Syscalls)

Ein Syscall ist wie ein roter Notfallknopf.
Das Programm drückt ihn, friert sich ein und ruft das OS zu Hilfe.

Notizen

Hardware-Interrupts sind nur die halbe Miete. User-Mode-Programme dürfen Monitor oder SSD nicht direkt anfassen — sie bitten das OS über Software-Traps um Hilfe.

Folie

61 / 84

Warum wir das OS brauchen

- Warum nicht einfach sw direkt auf den Monitor?
- Sicherheit: freier MMIO-Zugriff könnte die Platte zerstören; User-Mode sperrt MMIO hardwareseitig
- Abstraktion: tausende Monitore/Treiber — das OS kapselt Hardware in einer universellen Schnittstelle

Notizen

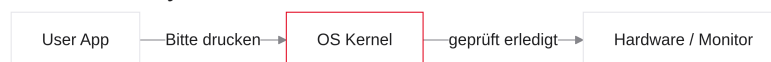
Betriebssysteme existieren wegen Sicherheit und Bequemlichkeit: `print` statt Grafikkarten-Handbuch.

Folie

62 / 84

Der Pförtner (Das Betriebssystem)

- OS im privilegierten Ring (S- oder M-Mode)
- Verwaltet Hardware exklusiv
- Agiert als Pförtner / Torwächter
- Drucken = Formular ausfüllen, an die Kernel-Tür klopfen
- Diese Formulare sind System Calls



Notizen

Türsteher vor der Hardware: Zettel (Syscall) prüfen, Treiber anstoßen, Kontrolle zurückgeben.

Folie

63 / 84

Der Befehl `ecall`

- RISC-V-Klopfen an die Kernel-Tür: `ecall` (Environment Call)
- Keine Operanden — steht nackt im Code
- Decoder liest `ecall` → sofort Trap-Handler
- Hardware-erzwungener Kontextwechsel

```
ecall    # aus U-Mode in den Kernel
```

Notizen

Instruction Decoder erkennt `ecall`, löst absichtlich einen Trap aus: PC nach `mtvec`, Kernel erwacht.

System Calls (Die API des OS)

- Hunderte Dienste: Datei, Netz, Zeit, Text drucken, ...
- Diese Liste = System Call API
- Frage nach `ecall`: Welcher Service?
- Antwort: Vertrag erfüllen — die Syscall-ABI

Notizen

Restaurant mit Speisekarte: Klingel (`ecall`) reicht nicht — Gericht muss vorher kommuniziert sein.

Die Syscall-ABI (Venus vs. Linux)

- Vor `ecall` muss eine Service-ID gesetzt werden — Registerkonvention ist ABI-abhängig
- Venus (Labor): ID in `a0`, Argumente in `a1`, `a2`, ...
- Linux / RARS: ID in `a7`, Argumente in `a0`, `a1`, ...
- Im Praktikum: Venus-Konvention verwenden

```
Venus:  a0 = Service-ID,  a1 = Argument
Linux:  a7 = Service-ID,  a0 = Argument
```

Notizen

Venus folgt der SPiM/MARS-Tradition (ID in `a0`). Echte RISC-V-Linux-Systeme und RARS nutzen `a7`. Verwechslung ist die häufigste Laborfalle.

Code-Beispiel: Print Integer (Venus)

- Service 1 in `a0`, Zahl 42 in `a1`, dann `ecall`
- Venus kehrt danach zur nächsten Instruktion zurück (Handler intern)

```
li a0, 1      # Venus: Service Print Integer
li a1, 42     # Argument
ecall
```

Notizen

Drei Zeilen unter Venus-ABI. Bare-Metal-Handler müssten zuvor `mepc += 4` setzen.

Code-Beispiel: Exit Program (Venus)

- Endlosschleife am Code-Ende ist unsauber
- Service 10 = Exit; keine Argumente; keine Rückkehr

```
li a0, 10      # Venus: Exit
ecall
```

Notizen

Professioneller Stil: nach dem Algorithmus Exit statt Endlosschleife.

Was macht ecall unter der Haube?

- Decoder liest `ecall` → Hardware schaltet:
 - PC des `ecall` → `mepc`
 - `mcause` = 8 (Environment Call from U-Mode)
 - PC → Adresse in `mtvec`
- Handler: Ursache 8 → Syscall; Service-ID laut ABI lesen; oft `mepc += 4`

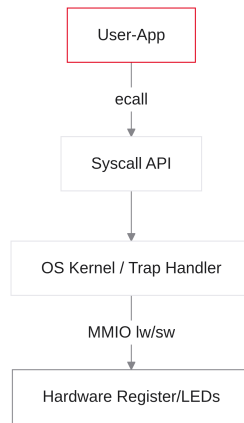


Notizen

Dieselbe Notaufnahme wie beim Hardware-Interrupt; Code 8 = absichtlicher User-Anruf. Venus verbirgt `mepc+4` und die Service-Auswertung.

Das Betriebssystem-Modell

- Schichten-Modell: Abstraktion schützt vor harter Physik



Notizen

App → Syscall-Vertrag → Kernel → nackte Hardware. Im Berufsleben trifft man oft alle Schichten.

Folie

70 / 84

Praxis: Ripes & Venus

Folie

71 / 84

Dualer Laboreinsatz

- Wechsel von Theorie zum Labor
- Ripes vs. Venus: unterschiedliche Zwecke
- Ripes: virtuelle MMIO-Hardware (LEDs)
- Venus: Konsole über `ecall`
- Typische I/O-Bugs und Clobbering-Warnung

Notizen

Theorie von Einheit 8 abgeschlossen: Bare-Metal und OS-Aufrufe mit beiden Werkzeugen.

Tool-Zweck im Labor

- Zwei Welten: nackte Hardware (MMIO) vs. Abstraktion (Syscalls)
- Ripes: Hardware-Simulator, Bare-Metal mit `lw/sw` auf MMIO
- Venus: OS-Simulator, fängt `ecall` ab und simuliert Konsole

```
Ripes  → Schalter-LED-Labor (Bare-Metal)
Venus  → Print-/EVA-Labor mit `.data` (Syscalls)
```

Notizen

Ripes = Physik ohne Linux; Venus = Kernel-Simulation mit Konsole.

Ripes I/O-Tab Setup

- Reiter I/O in Ripes öffnen
- Plus: Switches und LED Matrix hinzufügen
- Ripes verdrahtet virtuell und vergibt MMIO-Adressen

Notizen

Steckbrett-Simulation inkl. Adress-Decoder.

Ripes: MMIO-Adressen prüfen

- Bauteil anklicken → Konfiguration unten links
- Base Address zwingend ablesen
- Typisch: Switches `0xF0000000`, LEDs `0xF0000020`
- Diese Adressen in Register laden für `lw/sw`

Gerät	Typ	Typische Adresse
Switches	Input (Bit-Map)	<code>0xF0000000</code>
LED Matrix	Output (Wort-Array)	<code>0xF0000020</code>

Notizen

Ohne Hausnummer ist der Code blind; Basisadresse oft in `s`-Register.

Folie

75/84

Labor-Aufgabe 1 (Ripes)

- Hello Hardware: Endlosschleife liest Switches
- Mit `and`-Maske prüfen: Schalter 3 an?
- Ja: erste LED rot (`0x00FF0000`); nein: schwarz (`0x00000000`)
- Offset der ersten LED: 0 (Base)

```
li s1, 0xF0000000    # Switches
li s2, 0xF0000020    # LED-Matrix
Loop:
    lw t0, 0(s1)
    andi t1, t0, 8     # Maske Bit 3
    beq t1, zero, Off
    li t2, 0x00FF0000  # Rot
    j Store
Off:
    li t2, 0
Store:
    sw t2, 0(s2)
    j Loop
```

Notizen

Einheit 3 (Bits) + Einheit 8 (I/O): nicht nur „ungleich 0“, sondern exakt Schalter 3 maskieren.

Folie

76/84

Labor-Aufgabe 2 (Ripes)

- Lauflicht (Knight-Rider): grüne LED wandert links → rechts
- Offset-Pointer startet bei 0; Delay-Loop für Sichtbarkeit
- Pointer += 4 für nächstes LED-Wort; am Ende zurücksetzen

```
# Lauflicht im I/O-Tab beobachten
```

Notizen

Jede LED = 32 Bit; Pointer um 4 Bytes erhöhen, sonst unsichtbar schnell.

Wechsel zu Venus (OS-Simulation)

- Ripes schließen, Venus öffnen; Konsole unten
- Venus fängt `ecall` ab, wertet Service-ID in `a0` aus, schreibt in die Konsole
- Keine harten MMIO-Adressen — Arbeit an das OS delegieren

Notizen

Paradigmenwechsel: vom Bare-Metal-Bus zur Syscall-API (Venus-ABI!).

Die Venus Syscall-Tabelle

- Essenzielle Services für das Labor (ID immer in `a0`):

a0 (ID)	Name	Argumente	Resultat
1	Print Integer	a1 = Zahl	Zahl auf Konsole
4	Print String	a1 = Adresse	Text auf Konsole
10	Exit	keine	Programm endet

Hinweis: Venus hat keinen interaktiven Read-Int (anders als RARS).

Notizen

Labor nutzt Print und Exit. Interaktive Eingabe gehört nicht zum Standard-Venus.

Labor-Aufgabe 3 (Venus)

- Hello World: `.data` mit `.string "Hello World!\n"`
- `la a1, ...` — Startadresse; `li a0, 4;ecall`
- Sauber beenden: `li a0, 10/ecall`

```
la a1, msg
li a0, 4
ecall
li a0, 10
ecall
```

Notizen

String-Adresse in a1, Service-ID 4 in a0 — Venus-ABI.

Folie

80/84

Labor-Aufgabe 4 (Venus)

- EVA ohne Tastatur: Wert in `.data` (z. B. `n: .word 21`)
- Laden, verdoppeln (Shift oder `add`), per Service 1 drucken
- Optional: Prompt-String per Service 4 vorher ausgeben
- Mit Service 10 beenden

```
lw t0, n
slli a1, t0, 1    # *2
li a0, 1
ecall
```

Notizen

EVA-Prinzip über Syscalls; Input kommt aus `.data`, nicht von der Konsole.

Folie

81/84

Debugging I/O (Best Practices)

- Ripes LED schwarz?
 - Basisadresse korrekt?
 - Memory-Tab bei `0xF0...`: Farbmuster sichtbar?
 - Im RAM, aber dunkel: Refresh; nicht im RAM: `sw/Adresse` prüfen
- Venus stürzt bei `ecall`?
 - ID in `a0`, Argument in `a1` (nicht `a7/a0` wie bei Linux/RARS)
 - String: Adresse mit `la` in `a1`; Integer: Wert in `a1`

Simulator als Spion: Register vor `ecall` prüfen.

Notizen

Memory-Tab verrät, ob `sw` traf; Venus: ID in `a0`, Payload in `a1`.

Clobbering bei `ecall` (Warnung)

- `ecall` = ABI-Funktionsaufruf in den Kernel
- Laut ABI darf der Kernel Caller-Saved (`t0–t6`, `a0–a7`) zerstören
- Venus erhält `t`-Register faktisch oft — das ist nicht garantiert und nicht portabel
- Schleifenzähler in `t0` + `Print` in der Schleife: auf anderen Systemen riskant
- Lösung: vor `ecall` auf Stack sichern oder `s0–s11` nutzen

Notizen

Klassische Falle: Venus wirkt „nett“ zu `t`-Registern, die ABI erlaubt aber Clobbering. Portabler Code hält Zähler in `s`-Registern bzw. sichert vor dem `Syscall`.

Zusammenfassung Einheit 8

- MMIO: Hardware als RAM-Adresse — ohne Sonderbefehle
- Flüchtigkeit: MMIO ändert sich von selbst (Caching-Gefahr)
- Polling vs. Interrupts: Busy Waiting vs. Türklingel
- CSR-System: sichere Trap-Register (`mcause`, `mepc`, ...)
- Syscalls (`ecall`): Brücke vom User-Mode zum Kernel

Von Transistoren bis zum Betriebssystem:
das Ökosystem der Computerarchitektur ist durchdrungen.

Notizen

Isolation durchbrochen: Bare-Metal-I/O und der Tanz CPU OS.

Q&A und Hands-On

- Ripes oder Venus je nach Aufgabe öffnen
- Syscall-Tabelle als Referenz
- Arrays: Pointer +4 pro Wort; Strings: Null-Byte beachten
- Viel Erfolg beim Bau der Brücke zur Außenwelt



Notizen

Letzte Hands-On-Phase: Lauflicht in Ripes, EVA-Syscalls in Venus. Viel Erfolg!